

FASTAPI & PYDANTIC · QUICK REFERENCE

30 FastAPI & Pydantic Q&A

Endpoints, models, decorators, Uvicorn, Starlette.
With real code snippets for every core concept.

30

Q&A Pairs

Swipe to start →

1

What is FastAPI?

Ans- A modern, high-performance Python web framework for building APIs, built on Starlette and Pydantic, with automatic docs and async support.

2

What is Pydantic?

Ans- A data validation and settings library that uses Python type hints to validate, parse, and serialize data at runtime.

3

Why use FastAPI over Flask?

Ans- FastAPI gives async support, automatic request validation via Pydantic, and interactive Swagger docs out of the box, with better raw performance.

4

What is Starlette in relation to FastAPI?

Ans- Starlette is the lightweight ASGI framework FastAPI is built on top of. It handles routing, middleware, and WebSockets under the hood.

5

What is Uvicorn?

Ans- An ASGI server that actually runs your FastAPI app, handling incoming requests asynchronously using Python's asyncio event loop.

6

How do you define a GET endpoint?

Ans- Decorate a function with `@app.get(path)`. FastAPI serializes the return value to JSON automatically.

```
@app.get("/items/{item_id}")
def read_item(item_id: int):
    return {"item_id": item_id}
```

7

How do you define a POST endpoint?

Ans- Decorate with `@app.post(path)` and accept a Pydantic model as a parameter to parse and validate the JSON body.

```
@app.post("/items")
def create_item(item: Item):
    return item
```

8

How do you define a Pydantic model?

Ans- Subclass `BaseModel` and declare fields with type hints. Pydantic handles parsing, validation, and serialization.

```
from pydantic import BaseModel
class Item(BaseModel):
    name: str
    price: float
```

9

What is a path parameter?

Ans- Declared in curly braces in the route path and typed as a function argument. FastAPI validates and converts it automatically.

```
@app.get("/users/{user_id}")
def get_user(user_id: int):
    return {"user_id": user_id}
```

10

What is a query parameter?

Ans- Any function argument not in the path and not a Pydantic model, with an optional default value.

```
@app.get("/search")
def search(q: str = "", limit: int = 10):
    return {"q": q, "limit": limit}
```

11

How does request body validation work?

Ans- Declare a Pydantic model as a parameter type. FastAPI parses the JSON body and raises a 422 error automatically on failure.

```
@app.post("/orders")
def create_order(order: Order):
    return order
```

12

What is dependency injection in FastAPI?

Ans- Use Depends() to inject reusable logic, like DB sessions or auth checks, into route functions without duplicating code.

```
def get_db():  
    return SessionLocal()  
@app.get("/users")  
def list_users(db=Depends(get_db)):  
    return db.query(User).all()
```

13

What does the @app.get decorator do?

Ans- Registers the function as the handler for GET requests at the given path, and adds it to FastAPI's OpenAPI schema automatically.

14

What does the @app.post decorator do?

Ans- Registers the function as the handler for POST requests at the given path, typically used to create a resource from a body.

15

How do you run a FastAPI app with Uvicorn?

Ans- Point Uvicorn at the module and the app instance. The `--reload` flag restarts the server automatically on code changes.

```
uvicorn main:app --reload
```

16

What is Swagger UI in FastAPI?

Ans- FastAPI auto-generates interactive API docs at `/docs` and `/redoc`, built directly from your route type hints and Pydantic models.

17

What is `response_model` in FastAPI?

Ans- Set `response_model` on the route decorator to control and validate exactly what shape of data gets returned to the client.

```
@app.get("/items/{id}", response_model=Item)
def get_item(id: int):
    return db_item
```

18

How do you set default values in Pydantic?

Ans- Assign a default value directly to the field. Fields without a default are required; fields with one become optional.

```
class Item(BaseModel):  
    name: str  
    in_stock: bool = True
```

19

What is Field() used for in Pydantic?

Ans- Field() adds validation constraints and metadata to a model field, such as minimum length, value ranges, or descriptions.

```
class Item(BaseModel):  
    price: float = Field(gt=0)  
    name: str = Field(max_length=50)
```

20

What is BaseModel in Pydantic?

Ans- The core class every schema inherits from. It gives automatic parsing, validation, serialization, and .model_dump() methods.

21

How do you add custom validation in Pydantic?

Ans- Use the `field_validator` decorator to add custom logic beyond basic type checks, like rejecting negative values.

```
@field_validator("age")
def check_age(cls, v):
    if v < 0:
        raise ValueError("invalid")
    return v
```

22

async def vs def in FastAPI routes?

Ans- `async def` suits non-blocking I/O-bound work like DB or API calls. Plain `def` routes run in a threadpool automatically, so both are safe.

23

How do you raise a custom error response?

Ans- Raise `HTTPException` with a status code and detail message to return a proper error instead of a generic 500.

```
from fastapi import HTTPException
if not item:
    raise HTTPException(404, "Not found")
```

24

What is middleware in FastAPI?

Ans- Code that runs before and after every request, such as logging or adding custom headers, registered with `@app.middleware`.

```
@app.middleware("http")
async def add_header(request, call_next):
    response = await call_next(request)
    response.headers["X-Time"] = "1ms"
    return response
```

25

How do you manage config with Pydantic?

Ans- Subclass `BaseSettings` from `pydantic-settings` to load and validate configuration from environment variables automatically.

```
from pydantic_settings import BaseSettings
class Settings(BaseSettings):
    db_url: str
    debug: bool = False
```

26

What is APIRouter and why use it?

Ans- APIRouter lets you split routes into separate modules, then include them in the main app to keep large projects organized.

```
router = APIRouter(prefix="/users")
@router.get("/")
def list_users():
    return []
app.include_router(router)
```

27

How do you set a custom success status code?

Ans- Pass `status_code` to the route decorator to set the default success response code, such as 201 for resource creation.

```
@app.post("/items", status_code=201)
def create_item(item: Item):
    return item
```

28

Pydantic v1 vs v2, what changed?

Ans- V2 rewrote its core in Rust for major performance gains, renamed validator to `field_validator`, and moved `BaseSettings` to `pydantic-settings`.

29

How do you enable CORS in FastAPI?

Ans- Add CORSMiddleware to the app with allowed origins, so browsers on other domains can call your API without being blocked.

```
from fastapi.middleware.cors import CORSMiddleware
app.add_middleware(
    CORSMiddleware, allow_origins=["*"])
```

30

How do you run FastAPI in production?

Ans- Use Gunicorn as a process manager running multiple Uvicorn worker processes, which load-balances requests across CPU cores.

```
gunicorn main:app -w 4 \
-k uvicorn.workers.UvicornWorker
```

That's a wrap.

30/30 FastAPI & Pydantic Q&A

Save this for your next backend interview,
and keep it handy while building your next API.

Follow for more, and comment
“FASTAPI” for the full breakdown



Ritesh Rai, GenAI Engineer

300K+ LinkedIn impressions · Top 1% Creator on Topmate